

Guía de implementación de la interfaz principal con Jetpack Compose

IU Digital Radio

Procedimiento de desarrollo verificación publicación e integración

Dato	Asignación
Responsable	Juan Pablo Gonzalez
Rama	feature/ui-compose
Módulo	Pantalla principal y componentes declarativos
Punto de partida	main actualizado con los contratos de estado
Integración	Pull Request desde feature/ui-compose hacia main
Verificación	12 pruebas unitarias y 4 pruebas instrumentadas aprobadas
Versión de la guía	17/09/2026

Uso de la guía Juan Pablo debe ejecutar, revisar y explicar cada paso. La imagen del resultado esperado sirve como referencia y no sustituye las capturas tomadas durante su propio trabajo.

1 Objetivo del módulo

Juan Pablo construye la interfaz principal de IU Digital Radio con Jetpack Compose. La pantalla recibe RadioUiState, representa sus valores y convierte cada interacción del usuario en una RadioAction. Esta separación permite que la interfaz se pruebe antes de conectar la cámara, la vibración y el reproductor de audio.

- Crear una pantalla desplazable que funcione en teléfonos.
- Mostrar la emisora seleccionada y el estado de reproducción.
- Mostrar las cinco emisoras en dos columnas sin desplazamiento horizontal.
- Permitir seleccionar emisoras y emitir acciones de reproducción y silencio.
- Reservar un panel para mostrar la fotografía recibida desde el módulo de cámara.
- Definir una paleta y tipografía consistentes en ui.theme.
- Comprobar la interfaz con una vista previa y pruebas instrumentadas.

2 Responsabilidad de Juan Pablo

Corresponde a Juan Pablo	Corresponde a otros módulos
RadioScreen y sus componentes Compose.	Catálogo y reglas de estado de Luisa Gomez.
Presentación de botones y emisoras.	Reproducción real con Media3 de Jorge Echavarría.
Panel visual de la fotografía.	Permisos cámara y vibración de Edwin Ruiz.
Paleta tipografía tema y vista previa.	RadioRoute e integración final de Juan Ardila.
Pruebas instrumentadas de la interfaz.	Documentación final y control de calidad de Yeison Padron.

Límite técnico Los componentes de Juan Pablo emiten acciones. No solicitan permisos, no abren la cámara y no reproducen una transmisión.

3 Diferencia entre ui y ui theme

El paquete ui contiene pantallas, componentes y contratos relacionados con lo que se muestra y con las acciones del usuario. El subpaquete ui.theme contiene colores, tipografía y configuración visual reutilizable. Theme no es otra pantalla; define el aspecto compartido por todas las pantallas.

Paquete	Contenido
ui	RadioScreen RadioScreenDemo RadioUiState RadioAction y RadioReducer.
ui/components	StationSelector PlaybackControls y PhotoPanel.
ui/theme	Color Theme y Type.

4 Preparación desde main

La guía comienza con main actualizado. Juan Pablo no debe recoger directamente la rama de otro integrante. Recibe los cambios aprobados desde main y los incorpora a feature/ui-compose.

1. Abrir la terminal en la carpeta raíz que contiene gradlew.bat y settings.gradle.kts.
2. Cambiar a main y descargar su versión más reciente.
3. Cambiar a feature/ui-compose.
4. Fusionar main dentro de feature/ui-compose.
5. Confirmar que la rama activa sea feature/ui-compose y compilar antes de editar.

```
git switch main
git pull origin main
git switch feature/ui-compose
git merge main
git branch --show-current

$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
.\gradlew.bat assembleDebug
```

Resultado esperado La terminal muestra feature/ui-compose y BUILD SUCCESSFUL. Si la rama sólo se trabajará localmente durante la práctica, git merge main utiliza el main local ya preparado.

5 Mapa de archivos

Archivo	Tipo de cambio	Finalidad
MainActivity.kt	Modificar	Mostrar la ruta de demostración.
ui/RadioScreenDemo.kt	Crear	Mantener estado temporal para revisar la interfaz.
ui/RadioScreen.kt	Crear	Componer la pantalla principal.
ui/components/StationSelector.kt	Crear	Mostrar las cinco emisoras en dos columnas.
ui/components/PlaybackControls.kt	Crear	Mostrar controles de reproducción y silencio.
ui/components/PhotoPanel.kt	Crear	Mostrar el espacio y la foto de cámara.
ui/theme/Color.kt	Modificar	Definir la paleta visual.
ui/theme/Theme.kt	Modificar	Aplicar esquemas claro y oscuro.
ui/theme/Type.kt	Modificar	Definir jerarquía tipográfica.
RadioScreenTest.kt	Crear	Verificar contenido e interacción.

6 Implementación paso a paso

Las rutas siguientes parten de la raíz del proyecto. El código mostrado coincide con el esqueleto compilado y probado en feature/ui-compose.

6.1 Conexión provisional en MainActivity

Ruta app/src/main/java/com/example/iudigitalradioapp/MainActivity.kt

MainActivity elimina la pantalla Hello Android y muestra RadioScreenDemo dentro del tema. La conexión está documentada como provisional porque Juan Ardila incorporará RadioRoute cuando existan los módulos de hardware y audio.

- setContent inicia la composición declarativa.
- IudigitalradioappTheme aplica colores y tipografía.
- RadioScreenDemo permite ejecutar la interfaz sin implementar efectos externos.

```
package com.example.iudigitalradioapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.iudigitalradioapp.ui.RadioScreenDemo
import com.example.iudigitalradioapp.ui.theme.IudigitalradioappTheme

/**
 * Punto de entrada de la aplicación.
 *
 * La conexión con [RadioScreenDemo] permite revisar localmente la interfaz
 * creada por el rol simulado de Juan Pablo Gonzalez. Juan Ardila reemplazará
 * esta conexión provisional por RadioRoute durante la integración final.
 */
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            IudigitalradioappTheme {
                RadioScreenDemo()
            }
        }
    }
}
```

6.2 Estado de demostración

Ruta app/src/main/java/com/example/iudigitalradioapp/ui/RadioScreenDemo.kt

RadioScreenDemo conserva un RadioUiState con remember y procesa las acciones mediante RadioReducer. Esta función permite comprobar las interacciones visuales mientras RadioRoute todavía no existe.

- mutableStateOf vuelve observable el estado para Compose.
- El operador by permite leer y asignar state directamente.
- Cada acción produce un estado nuevo sin abrir cámara ni reproducir audio.

```
package com.example.iudigitalradioapp.ui

import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue

/**
 * Rol responsable en la simulación: Juan Pablo Gonzalez.
 *
 * * Mantiene estado únicamente para demostrar y revisar la interfaz. Esta
 * * función no reproduce audio, no abre la cámara y no activa vibración. Juan
 * * Ardila conectará esos efectos posteriormente desde RadioRoute.
 */
@Composable
fun RadioScreenDemo() {
    var state by remember {
        mutableStateOf(createInitialRadioUiState())
    }

    RadioScreen(
        state = state,
        onAction = { action ->
            state = RadioReducer.reduce(state, action)
        }
    )
}
```

6.3 Pantalla principal

Ruta app/src/main/java/com/example/iudigitalradioapp/ui/RadioScreen.kt

RadioScreen es una función sin estado propio. Recibe el estado completo y una función onAction. LazyColumn permite desplazar el contenido en pantallas pequeñas.

- Scaffold aplica correctamente el espacio del sistema.
- RadioHeader y SelectedStationCard resumen el estado actual.
- Los componentes secundarios reciben sólo los datos y eventos que necesitan.
- La vista previa usa datos reales del repositorio sin ejecutar la aplicación.

```
package com.example.iudigitalradioapp.ui

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.material3.ElevatedCard
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.iudigitalradioapp.data.StationRepository
import com.example.iudigitalradioapp.ui.components.PhotoPanel
import com.example.iudigitalradioapp.ui.components.PlaybackControls
import com.example.iudigitalradioapp.ui.components.StationSelector
import com.example.iudigitalradioapp.ui.theme.IudigitalradioappTheme

/**
 * Rol responsable en la simulación: Juan Pablo Gonzalez.
 *
 * Dibuja la pantalla principal a partir de [RadioUiState] y comunica cada
 * interacción mediante [RadioAction]. Al ser una función sin estado propio,
 * puede previsualizarse, probarse y conectarse después con RadioRoute.
 */
@Composable
fun RadioScreen(
    state: RadioUiState,
    onAction: (RadioAction) -> Unit,
    modifier: Modifier = Modifier
) {
    Scaffold(modifier = modifier.fillMaxSize()) { innerPadding ->
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding),
            contentPadding = PaddingValues(horizontal = 20.dp, vertical = 24.dp),
            verticalArrangement = Arrangement.spacedBy(20.dp)
        ) {
            item {
                RadioHeader(isPlaying = state.isPlaying)
            }

            item {
                SelectedStationCard(state = state)
            }

            item {
                StationSelector(
                    stations = state.stations,
                    selectedStationId = state.selectedStationId,

```

```

        onStationSelected = { stationId ->
            onAction(RadioAction.SelectStation(stationId))
        }
    )
}

item {
    PlaybackControls(
        hasSelectedStation = state.selectedStation != null,
        isPlaying = state.isPlaying,
        isMuted = state.isMuted,
        onPlayPause = {
            onAction(
                if (state.isPlaying) RadioAction.Pause else RadioAction.Play
            )
        },
        onToggleMute = {
            onAction(RadioAction.ToggleMute)
        }
    )
}

item {
    PhotoPanel(
        photo = state.capturedPhoto,
        onOpenCamera = {
            onAction(RadioAction.OpenCamera)
        }
    )
}
}

}

}

/** Encabezado visual creado por Juan Pablo Gonzalez. */
@Composable
private fun RadioHeader(
    isPlaying: Boolean,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxWidth(),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        Text(
            text = "IU Digital Radio",
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold
        )
        Text(
            text = "Emisoras universitarias, música y actualidad",
            style = MaterialTheme.typography.bodyLarge,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
        Surface(
            color = if (isPlaying) {
                MaterialTheme.colorScheme.tertiaryContainer
            } else {
                MaterialTheme.colorScheme.surfaceVariant
            },
            contentColor = if (isPlaying) {
                MaterialTheme.colorScheme.onTertiaryContainer
            } else {
                MaterialTheme.colorScheme.onSurfaceVariant
            },
            shape = MaterialTheme.shapes.extraLarge
        ) {
            Text(
                text = if (isPlaying) "EN REPRODUCCIÓN" else "EN PAUSA",
                modifier = Modifier.padding(horizontal = 12.dp, vertical = 6.dp),
                style = MaterialTheme.typography.labelMedium,
                fontWeight = FontWeight.Bold
            )
        }
    }
}

```

```

    }
}

/** Tarjeta de selección actual creada por Juan Pablo Gonzalez. */
@Composable
private fun SelectedStationCard(
    state: RadioUiState,
    modifier: Modifier = Modifier
) {
    val station = state.selectedStation

    ElevatedCard(modifier = modifier.fillMaxWidth()) {
        Column(
            modifier = Modifier.padding(20.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            Text(
                text = "Emisora seleccionada",
                style = MaterialTheme.typography.labelLarge,
                color = MaterialTheme.colorScheme.primary
            )
            Text(
                text = station?.name ?: "Selecciona una emisora",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.Bold
            )
            Text(
                text = station?.description
                    ?: "El catálogo no contiene una selección disponible.",
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }
}

@Preview(showBackground = true, widthDp = 390, heightDp = 844)
@Composable
private fun RadioScreenPreview() {
    IudigitalradioappTheme(dynamicColor = false) {
        RadioScreen(
            state = createInitialRadioUiState(StationRepository.stations),
            onAction = {}
        )
    }
}

```


6.4 Selector de emisoras

Ruta app/src/main/java/com/example/iudigitalradioapp/ui/components/StationSelector.kt

StationSelector distribuye el catálogo en filas de dos columnas. Cada FilterChip compara su id con selectedStationId y devuelve el identificador elegido. La quinta emisora queda centrada con el ancho de una columna.

- chunked(2) divide la lista en grupos de dos emisoras.
- Column y Row muestran todas las opciones sin desplazamiento horizontal.
- StationChip evita repetir la construcción y el evento de cada FilterChip.
- El componente muestra un mensaje cuando la lista está vacía.
- No modifica StationRepository ni conoce RadioReducer.

```
package com.example.iudigitalradioapp.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.material3.FilterChip
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.iudigitalradioapp.model.Station

/**
 * Rol responsable en la simulación: Juan Pablo Gonzalez.
 *
 * Presenta las cinco emisoras en una cuadrícula de dos columnas para que el
 * usuario pueda verlas sin desplazarse horizontalmente. Devuelve únicamente
 * el identificador seleccionado y no modifica el repositorio.
 */
@Composable
fun StationSelector(
    stations: List<Station>,
    selectedStationId: String?,
    onStationSelected: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxWidth(),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        Text(
            text = "Explorar emisoras",
            style = MaterialTheme.typography.titleMedium
        )

        if (stations.isEmpty()) {
            Text(
                text = "No hay emisoras disponibles.",
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        } else {
            Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                stations.chunked(2).forEach { stationRow ->
                    if (stationRow.size == 1) {
                        Box(
                            modifier = Modifier.fillMaxWidth(),
                            contentAlignment = Alignment.Center
                        ) {
                            StationChip(
                                station = stationRow.first(),

```

```

                selectedStationId = selectedStationId,
                onStationSelected = onStationSelected,
                modifier = Modifier.fillMaxWidth(0.5f)
            )
        }
    } else {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(8.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            stationRow.forEach { station ->
                StationChip(
                    station = station,
                    selectedStationId = selectedStationId,
                    onStationSelected = onStationSelected,
                    modifier = Modifier.weight(1f)
                )
            }
        }
    }
}

/** Botón reutilizable de una emisora dentro de la cuadrícula de Juan Pablo. */
@Composable
private fun StationChip(
    station: Station,
    selectedStationId: String?,
    onStationSelected: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    FilterChip(
        selected = station.id == selectedStationId,
        onClick = {
            onStationSelected(station.id)
        },
        label = {
            Text(text = station.name)
        },
        modifier = modifier
    )
}

```

6.5 Controles de reproducción

Ruta app/src/main/java/com/example/iudigitalradioapp/ui/components/PlaybackControls.kt

PlaybackControls adapta el texto de sus botones al estado recibido. Los botones se deshabilitan si no existe una emisora seleccionada.

- El botón principal alterna entre Reproducir y Pausar.
- El botón tonal alterna entre Silenciar y Activar sonido.
- Los callbacks comunican intención sin depender de Media3.

```

package com.example.iudigitalradioapp.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.material3.Button
import androidx.compose.material3.FilledTonalButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier

```

```
import androidx.compose.ui.unit.dp

/**
 * Rol responsable en la simulación: Juan Pablo Gonzalez.
 *
 * Muestra los controles y emite intenciones. La reproducción y el volumen
 * reales se conectarán posteriormente con el módulo de audio.
 */
@Composable
fun PlaybackControls(
    hasSelectedStation: Boolean,
    isPlaying: Boolean,
    isMuted: Boolean,
    onPlayPause: () -> Unit,
    onToggleMute: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxWidth(),
        verticalArrangement = Arrangement.spacedBy(10.dp)
    ) {
        Text(
            text = "Controles",
            style = MaterialTheme.typography.titleMedium
        )
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            Button(
                onClick = onPlayPause,
                enabled = hasSelectedStation,
                modifier = Modifier.weight(1f)
            ) {
                Text(text = if (isPlaying) "Pausar" else "Reproducir")
            }
            FilledTonalButton(
                onClick = onToggleMute,
                enabled = hasSelectedStation,
                modifier = Modifier.weight(1f)
            ) {
                Text(text = if (isMuted) "Activar sonido" else "Silenciar")
            }
        }
    }
}
```

6.6 Panel de fotografía

Ruta app/src/main/java/com/example/iudigitalradioapp/ui/components/PhotoPanel.kt

PhotoPanel muestra un espacio informativo cuando no existe fotografía y representa el Bitmap cuando el estado contiene una captura.

- aspectRatio mantiene el área visual estable.
- ContentScale.Crop adapta la fotografía al contenedor.
- Abrir cámara sólo ejecuta onOpenCamera; Edwin implementará el efecto real.

```
package com.example.iudigitalradioapp.ui.components

import android.graphics.Bitmap
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.aspectRatio
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.shape.RoundedCornerShape
```

```

import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.unit.dp

/**
 * Rol responsable en la simulación: Juan Pablo Gonzalez.
 *
 * Reserva el espacio visual de la cámara y muestra el Bitmap recibido. Edwin
 * Ruiz implementará el permiso, la captura real y la vibración asociada.
 */
@Composable
fun PhotoPanel(
    photo: Bitmap?,
    onOpenCamera: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxWidth(),
        verticalArrangement = Arrangement.spacedBy(10.dp)
    ) {
        Text(
            text = "Foto de la experiencia",
            style = MaterialTheme.typography.titleMedium
        )

        if (photo == null) {
            Box(
                modifier = Modifier
                    .fillMaxWidth()
                    .aspectRatio(16f / 9f)
                    .clip(RoundedCornerShape(20.dp))
                    .background(MaterialTheme.colorScheme.surfaceVariant)
                    .padding(24.dp),
                contentAlignment = Alignment.Center
            ) {
                Text(
                    text = "La fotografía aparecerá aquí después de usar la cámara.",
                    style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        } else {
            Image(
                bitmap = photo.asImageBitmap(),
                contentDescription = "Fotografía capturada",
                modifier = Modifier
                    .fillMaxWidth()
                    .aspectRatio(16f / 9f)
                    .clip(RoundedCornerShape(20.dp)),
                contentScale = ContentScale.Crop
            )
        }

        Button(
            onClick = onOpenCamera,
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Abrir cámara")
        }
    }
}

```

6.7 Paleta de colores

Ruta app/src/main/java/com/example/iudigitalradioapp/ui/theme/Color.kt

Color.kt reemplaza los colores de plantilla por una paleta azul naranja y neutra apropiada para la interfaz de radio.

- Los nombres describen la función visual del color.
- Se incluyen superficies claras y oscuras.

```
package com.example.iudigitalradioapp.ui.theme

import androidx.compose.ui.graphics.Color

/** Paleta visual definida por el rol simulado de Juan Pablo Gonzalez. */
val RadioBlue = Color(0xFF2457C5)
val RadioBlueLight = Color(0xFFB7C9FF)
val RadioNavy = Color(0xFF10233F)
val RadioNavyLight = Color(0xFFD6E3FF)
val RadioOrange = Color(0xFFF28C28)
val RadioOrangeDark = Color(0xFF8A4D00)

val RadioBackground = Color(0xFFF7F9FC)
val RadioSurface = Color(0xFFFFFFFF)
val RadioSurfaceVariant = Color(0xFFE8EDF5)
val RadioDarkBackground = Color(0xFF0E1726)
val RadioDarkSurface = Color(0xFF172338)
```

6.8 Tema de la aplicación

Ruta app/src/main/java/com/example/iudigitalradioapp/ui/theme/Theme.kt

Theme.kt relaciona la paleta con los roles de Material 3. El color dinámico permanece desactivado para que las capturas se vean iguales en distintos dispositivos.

- LightColorScheme define la apariencia normal.
- DarkColorScheme conserva contraste en modo oscuro.
- MaterialTheme entrega el esquema y la tipografía a toda la composición.

```
package com.example.iudigitalradioapp.ui.theme

import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color

private val DarkColorScheme = darkColorScheme(
    primary = RadioBlueLight,
    onPrimary = RadioNavy,
    secondary = RadioNavyLight,
    tertiary = RadioOrange,
    onTertiary = Color.Black,
    background = RadioDarkBackground,
    surface = RadioDarkSurface
)

private val LightColorScheme = lightColorScheme(
    primary = RadioBlue,
    onPrimary = Color.White,
    secondary = RadioNavy,
    onSecondary = Color.White,
    tertiary = RadioOrangeDark,
    background = RadioBackground,
    surface = RadioSurface,
    surfaceVariant = RadioSurfaceVariant,
    onSurface = RadioNavy
)

/** Tema visual mantenido por el rol simulado de Juan Pablo Gonzalez. */
@Suppress("UNUSED_PARAMETER")
@Composable
fun IudigitalradioappTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = false,
    content: @Composable () -> Unit
) {
    // La paleta se mantiene estable para que la evidencia sea consistente.
    val colorScheme = if (darkTheme) DarkColorScheme else LightColorScheme

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

6.9 Tipografía

Ruta app/src/main/java/com/example/iudigitalradioapp/ui/theme/Type.kt

Type.kt define tamaños y pesos para encabezados títulos cuerpo y etiquetas. Los componentes consumen estos estilos mediante MaterialTheme.typography.

- headlineMedium se usa en el nombre de la aplicación.

- titleLarge y titleMedium ordenan la información de las secciones.
- labelMedium se usa en el indicador de estado.

```
package com.example.iudigitalradioapp.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

// Set of Material typography styles to start with
val Typography = Typography(
    headlineMedium = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Bold,
        fontSize = 28.sp,
        lineHeight = 34.sp
    ),
    titleLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.SemiBold,
        fontSize = 22.sp,
        lineHeight = 28.sp
    ),
    titleMedium = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.SemiBold,
        fontSize = 18.sp,
        lineHeight = 24.sp
    ),
    bodyLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.5.sp
    ),
    labelMedium = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Medium,
        fontSize = 12.sp,
        lineHeight = 16.sp,
        letterSpacing = 0.5.sp
    )
)
```

6.10 Pruebas de interfaz

Ruta app/src/androidTest/java/com/example/iudigitalradioapp/ui/RadioScreenTest.kt

RadioScreenTest abre MainActivity en un emulador. Una prueba verifica el contenido principal, otra comprueba que las cinco emisoras estén visibles sin desplazamiento horizontal y la tercera pulsa Reproducir para confirmar el cambio a Pausar y EN REPRODUCCIÓN.

- createAndroidComposeRule inicia la actividad real.
- onNodeWithText localiza elementos por su semántica.
- assertCountEquals controla el nombre repetido de la emisora seleccionada.
- performClick reproduce una interacción del usuario.

```
package com.example.iudigitalradioapp.ui

import androidx.compose.ui.test.assertCountEquals
import androidx.compose.ui.test.assertIsDisplayed
import androidx.compose.ui.test.junit4.createAndroidComposeRule
import androidx.compose.ui.test.onAllNodesWithText
import androidx.compose.ui.test.onNodeWithText
import androidx.compose.ui.test.performClick
import com.example.iudigitalradioapp.MainActivity
import org.junit.Rule
import org.junit.Test

/** Pruebas de interfaz del rol simulado de Juan Pablo Gonzalez. */
class RadioScreenTest {

    @get:Rule
    val composeRule = createAndroidComposeRule<MainActivity>()

    @Test
    fun selectedStationAndControlsAreVisible() {
        composeRule.onNodeWithText("Radio universitaria - Drexel University").assertIsDisplayed()
        composeRule.onNodeWithText("Reproducir").assertIsDisplayed()
        composeRule.onNodeWithText("Abrir cámara").assertExists()
    }

    @Test
    fun allFiveStationsAreVisibleWithoutHorizontalScroll() {
        composeRule.onAllNodesWithText("WKDU Philadelphia 91.7 FM")
            .assertCountEquals(2)[0]
            .assertIsDisplayed()

        listOf(
            "KEXP",
            "NUCROOZE Radio",
            "Groove Salad",
            "Indie Pop Rocks!"
        ).forEach { stationName ->
            composeRule.onNodeWithText(stationName).assertIsDisplayed()
        }
    }

    @Test
    fun playButtonUpdatesVisibleState() {
        composeRule.onNodeWithText("Reproducir").performClick()

        composeRule.onNodeWithText("Pausar").assertIsDisplayed()
        composeRule.onNodeWithText("EN REPRODUCCIÓN").assertIsDisplayed()
    }
}
```


7 Resultado esperado

La aplicación debe mostrar la emisora seleccionada, las cinco opciones en una cuadrícula de dos columnas, los controles y el panel de fotografía. No se requiere un gesto horizontal para descubrir emisoras. El contenido general puede desplazarse verticalmente para mantener accesible el botón de cámara.



Figura 1 Resultado esperado del esqueleto Compose en un emulador Pixel 4 XL

Importante Esta figura es una referencia de revisión. Juan Pablo debe tomar sus propias capturas mientras crea ejecuta y explica la interfaz.

8 Verificación técnica

La verificación se realiza en tres niveles. Primero se compila la aplicación y las pruebas. Después se ejecutan las pruebas unitarias. Por último se ejecutan las pruebas Compose en un emulador o dispositivo conectado.

```
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
.\gradlew.bat testDebugUnitTest assembleDebug compileDebugAndroidTestKotlin
```

Con el emulador iniciado y desbloqueado:

```
.\gradlew.bat connectedDebugAndroidTest
```

Resultado verificado BUILD SUCCESSFUL. Se aprobaron 12 pruebas unitarias y 4 pruebas instrumentadas: 3 de RadioScreenTest y 1 prueba base del proyecto.

8.1 Solución cuando no aparece la jerarquía Compose

Si el informe muestra No compose hierarchies found in the app, comprobar primero que el emulador esté encendido y desbloqueado. En la verificación local el fallo se produjo mientras el emulador estaba dormido y desapareció al despertarlo.

```
adb devices
adb shell input keyevent KEYCODE_WAKEUP
adb shell wm dismiss-keyguard
.\gradlew.bat connectedDebugAndroidTest
```

9 Revisión y commit local

Juan Pablo debe revisar sólo sus archivos. No debe agregar .idea/misc.xml output tmp ni otros archivos locales. Se usan rutas específicas en lugar de git add punto.

```
git status --short --branch
git diff --check

git add app/src/main/java/com/example/iudigitalradioapp/MainActivity.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/RadioScreen.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/RadioScreenDemo.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/components/StationSelector.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/components/PlaybackControls.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/components/PhotoPanel.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/theme/Color.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/theme/Theme.kt
git add app/src/main/java/com/example/iudigitalradioapp/ui/theme/Type.kt
git add app/src/androidTest/java/com/example/iudigitalradioapp/ui/RadioScreenTest.kt

git diff --cached
git commit -m "Crea interfaz principal con Jetpack Compose"
```

Commit comprobado El esqueleto quedó registrado localmente como 08300b7. La rama aparece dos commits por delante del remoto porque contiene el commit de estado recibido desde main y el nuevo commit de interfaz.

10 Copia ZIP opcional

El ZIP sirve como respaldo del contenido registrado en HEAD. No reemplaza la rama ni el Pull Request.

```
git archive --format=zip --output ..\IU_Digital_Radio_Juan_Pablo_Gonzalez.zip HEAD
```

11 Publicación de la rama

Cuando la revisión local termine y main remoto ya contenga el módulo anterior aprobado, Juan Pablo publica únicamente feature/ui-compose.

```
git switch feature/ui-compose
git status --short --branch
git push origin feature/ui-compose
```

No usar push directo Juan Pablo no ejecuta git push origin main. El trabajo llega a main mediante un Pull Request revisado.

12 Pull Request hacia main

1. Abrir el repositorio en GitHub y seleccionar Compare and pull request.
2. Confirmar base main y compare feature/ui-compose.
3. Usar el título Crea interfaz principal con Jetpack Compose.
4. Describir los componentes creados las pruebas y los límites del módulo.
5. Solicitar la revisión de Juan Ardila.
6. Fusionar sólo cuando el proyecto compile y la revisión esté aprobada.

```
Resumen
- Reemplaza la pantalla inicial por IU Digital Radio.
- Agrega RadioScreen y componentes reutilizables.
- Muestra las cinco emisoras en dos columnas sin desplazamiento horizontal.
- Conecta temporalmente el estado mediante RadioScreenDemo.
- Define colores tipografía y tema Material 3.
- Agrega pruebas instrumentadas de contenido e interacción.

Verificación
- testDebugUnitTest assembleDebug
- connectedDebugAndroidTest
- 12 pruebas unitarias y 4 instrumentadas aprobadas.

Pendientes de integración
- RadioRoute reemplazará RadioScreenDemo.
- Cámara vibración y audio real serán conectados por sus responsables.
```

13 Confirmación final en main

Después de aprobar el Pull Request, el trabajo de Juan Pablo queda incorporado en main. Para comprobarlo localmente:

```
git switch main
git pull --ff-only origin main
.\gradlew.bat testDebugUnitTest assembleDebug
```

Fin del flujo Juan Pablo empieza desde main actualizado trabaja en feature/ui-compose publica esa rama y termina con su módulo integrado en main mediante revisión.

14 Capturas recomendadas

Número	Captura	Qué demuestra
1	main actualizado y cambio a feature/ui-compose	La rama parte de una base aprobada.
2	Árbol de paquetes ui components y ui theme	La interfaz está organizada por responsabilidades.
3	RadioScreen.kt en el editor	La pantalla recibe estado y emite acciones.
4	StationSelector con cinco emisoras	La cuadrícula presenta dos columnas sin gesto horizontal.
5	PhotoPanel.kt	La UI está preparada para recibir una fotografía.
6	Vista previa Compose	La pantalla puede revisarse sin ejecutar efectos externos.
7	Aplicación ejecutada en el emulador	Las cinco emisoras aparecen sin desplazamiento horizontal.
8	Botón en estado Pausar	La interfaz responde a una acción y recompone.
9	Pruebas instrumentadas aprobadas	El contenido y la interacción fueron verificados.
10	Commit 08300b7	Los archivos de Juan Pablo quedaron registrados.
11	Pull Request feature/ui-compose hacia main	El cambio fue enviado a revisión.
12	main después de la fusión	El módulo terminó integrado.

Ejemplo de pie de figura: Figura 3. Creación de RadioScreen como función sin estado. Nota. Elaboración propia a partir del proyecto IU Digital Radio.

15 Lista de cierre

- [] Actualicé main antes de trabajar y confirmé feature/ui-compose como rama activa.
- [] Puedo explicar por qué RadioScreen recibe estado y onAction.
- [] Puedo distinguir ui components y ui.theme.
- [] Las cinco emisoras están visibles sin desplazamiento horizontal.
- [] La selección reproducción pausa y silencio cambian visualmente.
- [] El botón de cámara sólo emite RadioAction.OpenCamera.
- [] La aplicación y las pruebas finalizan con BUILD SUCCESSFUL.
- [] Excluí .idea output tmp y archivos personales del commit.
- [] Publiqué feature/ui-compose y abrí el Pull Request hacia main.
- [] Verifiqué main después de la fusión.

16 Preguntas para explicar el trabajo

- ¿Qué diferencia existe entre RadioScreen y RadioScreenDemo?
- ¿Por qué RadioScreen no usa remember para guardar su propio estado?
- ¿Por qué RadioScreen usa LazyColumn y StationSelector usa Column y Row?
- ¿Por qué los componentes emiten callbacks en lugar de reproducir audio directamente?
- ¿Qué ocurre cuando RadioUiState cambia después de pulsar Reproducir?
- ¿Por qué ui.theme es diferente del paquete ui aunque esté dentro de él?
- ¿Qué diferencia existe entre push de la rama y fusión del Pull Request en main?

17 Referencias técnicas

Android Developers. (s. f.). Compose layout basics.

<https://developer.android.com/develop/ui/compose/layouts/basics>

Android Developers. (s. f.). State and Jetpack Compose.

<https://developer.android.com/develop/ui/compose/state>

Android Developers. (s. f.). Test your Compose layout.

<https://developer.android.com/develop/ui/compose/testing>

IU Digital. (s. f.). Evidencia de aprendizaje 3 Aplicación móvil Android IU Digital Radio Guía de orientación.

Las referencias del informe final deben completarse con los datos bibliográficos disponibles en el aula y presentarse según APA séptima edición.